API externe et API internes : IDS Comportemental

Ce document décrit le service externe utilisé par le système (AbuseIPDB) ainsi que les points de terminaison de l'API interne Flask, avec leurs formats d'entrée et de sortie.

1. API externe : AbuseIPDB

Rôle. Lorsqu'une alerte concerne une adresse IP publique, le backend interroge AbuseIPDB pour savoir si cette IP est connue pour des activités malveillantes. La réponse enrichit l'alerte : un comportement jugé anormal par le modèle sur une IP déjà signalée par de nombreux utilisateurs est plus crédible.

Élément	Détail
Service	AbuseIPDB (réputation d'adresses IP)
Requête	GET https://api.abuseipdb.com/api/v2/check
Authentification	Clé API envoyée dans l'en-tête « Key » de la requête
Entrée	ipAddress : l'adresse IP à vérifier ; maxAgeInDays : ancienneté maximale des signalements pris en compte
Sortie utilisée	abuseConfidenceScore (0 à 100), countryCode, isp
Erreurs gérées	Clé invalide (401), quota dépassé (429), service indisponible ou délai dépassé

Exemple de réponse (champs utiles)

```
{
  "data": {
    "ipAddress": "185.220.101.5",
    "abuseConfidenceScore": 95,
    "countryCode": "DE",
    "isp": "Exemple Hosting GmbH"
  }
}
```

Règles d'intégration

1. Fonctionnalité optionnelle. Si le service est en panne ou si le quota est dépassé, l'alerte est quand même enregistrée, sans score de réputation. Le système ne dépend jamais d'un service externe pour fonctionner.
2. Uniquement les IP publiques. Les adresses locales (192.168.x.x, 10.x.x.x, 172.16.x.x) ne sont jamais envoyées au service.
3. Clé API protégée. Elle est stockée dans une variable d'environnement, jamais dans le code source ni dans le dépôt Git.
4. Quota limité. L'offre gratuite limite le nombre de requêtes par jour (à vérifier sur le site du service). Les résultats sont gardés en mémoire pour ne pas redemander la même IP.
5. Confidentialité. Des IP publiques sont transmises à un tiers. C'est la raison pour laquelle cette fonction reste facultative et désactivable.

Impact sur la base de données

Deux champs facultatifs sont ajoutés à la table **alerts** : **reputation_score** (entier, peut être vide) et **country** (texte, peut être vide).

2. API internes (Flask)

L'API interne est utilisée par le dashboard web pour lire les données et agir sur les alertes. Toutes les réponses sont au format JSON. La capture réseau et la détection ne passent pas par cette API : elles s'exécutent dans le même processus que le backend et écrivent directement en base via AlertRepository.

Méthode	Endpoint	Rôle
GET	/api/alerts	Lister les alertes (filtres et tri)
GET	/api/stats	Métriques des cartes et données du graphique
PATCH	/api/alerts/{id}	Changer le statut d'une alerte

2.1 GET /api/alerts

Renvoie la liste des alertes, de la plus récente à la plus ancienne par défaut.

Paramètre	Obligatoire	Valeurs	Description
severity	Non	high, medium, low	Filtre par gravité
status	Non	new, treated, false_positive	Filtre par statut
sort	Non	date_desc (défaut), date_asc	Ordre de tri par date
limit	Non	entier (défaut : 50)	Nombre maximum d'alertes renvoyées

Exemple de réponse (200)

```
{
  "alerts": [
    {
      "id": 1,
      "timestamp": "2026-09-28T14:32:10",
      "src_ip": "192.168.1.47",
      "dst_ip": "10.0.0.5",
      "anomaly_type": "Scan port",
      "score": 0.92,
      "severity": "high",
      "status": "new",
      "reputation_score": null,
      "country": null
    }
  ],
  "total": 1
}
```

Gravité. Le champ severity n'est pas stocké en base : il est calculé à partir du score (score ≥ 0,8 : high ; entre 0,5 et 0,8 : medium ; sous 0,5 : low). **Erreurs.** 400 si un paramètre est invalide ; 500 en

cas d'erreur serveur.

2.2 GET /api/stats

Renvoie les chiffres des quatre cartes du dashboard et les points du graphique de trafic. Aucun paramètre.

Exemple de réponse (200)

```
{
  "threat_level": "moderate",
  "alerts_24h": 6,
  "packets_analyzed": 128430,
  "active_hosts": 14,
  "traffic": [
    { "timestamp": "14:00", "packet_count": 320 },
    { "timestamp": "14:05", "packet_count": 385 }
  ]
}
```

Sources. traffic_stats (graphique), alerts (compteur sur 24 h), hosts (hôtes actifs). **Erreur.** 500 en cas d'erreur serveur.

2.3 PATCH /api/alerts/{id}

Modifie le statut d'une alerte, par exemple lorsque l'administrateur clique sur « Traiter ». Valeurs acceptées : new, treated, false_positive.

Corps de la requête

```
{ "status": "treated" }
```

Exemple de réponse (200)

```
{ "id": 1, "status": "treated" }
```

Code	Cas
200	Statut modifié avec succès
400	Statut absent ou valeur non autorisée
404	Aucune alerte avec cet identifiant
500	Erreur serveur

3. Lien avec les diagrammes de séquence

Le chargement du dashboard correspond à GET /api/alerts et GET /api/stats (diagramme n°2). Le bouton « Traiter » correspond à PATCH /api/alerts/{id} (diagramme n°3). L'appel à AbuseIPDB intervient dans le pipeline de détection (diagramme n°1), juste avant l'enregistrement de l'alerte.